

BatchIt

Multi-Chain Batch Transfer Protocol

Technical White Paper

Version 1.0 · September 2026

[Celo Mainnet](#) · [Base Mainnet](#)

Abstract

BatchIt is a decentralized, multi-chain batch token transfer platform that enables users to distribute cryptocurrency — both ERC-20 tokens and native coins — to an arbitrary number of recipients within a single on-chain transaction. By collapsing what would otherwise be dozens or hundreds of individual transfers into one atomic contract call, BatchIt drastically reduces gas expenditure, shortens execution time, and simplifies the bookkeeping burden for projects, DAOs, payroll systems, and token-distribution campaigns.

The platform is built as a React + TypeScript single-page application backed by audited smart contracts deployed on Celo Mainnet and Base Mainnet. Wallet integration is handled via the MetaMask EIP-1193 provider through the Web3.js library, with full ERC-20 allowance management handled transparently before every token batch. Recipients can be entered manually, or imported in bulk via CSV, TXT, or Excel (XLSX) files, making the tool approachable for non-technical finance operators as well as blockchain engineers.

This white paper covers: the problem BatchIt solves, its architecture, the smart-contract interface it targets, the three transfer modes it offers, its file-import capability, supported networks and tokens, security considerations, and the user journey from wallet connection through transaction confirmation.

Table of Contents

- 1. Introduction & Motivation**
- 2. Protocol Overview**
- 3. Smart Contract Architecture**
- 4. Supported Networks & Tokens**
- 5. Transfer Modes**
- 6. File Import System**
- 7. Frontend Architecture**
- 8. User Journey**
- 9. Security Model**
- 10. Gas Economics**
- 11. Use Cases**
- 12. Roadmap & Future Work**
- 13. Conclusion**

1. Introduction & Motivation

Distributing tokens on a blockchain is one of the most common operations in decentralized finance: paying contributors, rewarding stakers, airdropping governance tokens, running marketing campaigns, or settling payroll for a DAO treasury. Yet the naive approach — issuing one transaction per recipient — is both expensive and operationally fragile.

1.1 The Problem with Sequential Transfers

Consider a project that needs to pay 200 contributors at end of month. Executing 200 individual ERC-20 transfers means:

- 200 separate MetaMask confirmation dialogs (or 200 programmatic wallet calls)
- 200 transactions competing for block space, each paying a base fee and a priority tip
- 200 transaction hashes to track for accounting
- Latency: even at one transaction every 5 seconds, 200 transfers takes over 16 minutes
- If any single transfer fails, the operator must manually identify and retry it

On a low-cost chain like Celo (average gas price ~ 0.001 GWEI, block time ~ 5 seconds) this is inconvenient. On a higher-cost chain like Base during congestion it becomes prohibitively expensive. BatchIt eliminates all of these pain points by encoding the entire distribution in a single transaction.

1.2 The BatchIt Solution

BatchIt compresses N token transfers into exactly one blockchain transaction by delegating the distribution logic to an on-chain smart contract. The caller sends one signed transaction; the contract iterates through the recipient list and dispatches each individual transfer. From the blockchain's perspective there is still one transaction (one transaction hash, one gas payment, one atomic outcome: it all succeeds or it all reverts).

From the user's perspective the flow is: connect wallet → pick network → pick token → enter or upload recipients → click Send. BatchIt handles allowance checking and auto-approval transparently.

2. Protocol Overview

BatchIt is structured as three tightly integrated layers:

2.1 Smart Contract Layer (On-chain)

A Solidity contract deployed at verified addresses on Celo and Base exposes three public functions: `batchTransfer` (ERC-20, variable amounts), `batchTransferEqual` (ERC-20, identical amounts), and `batchTransferETH` (native coin, variable amounts). The contract is stateless and non-upgradeable by default — it holds no funds and exercises no administrative authority over the caller's assets.

2.2 Application Layer (Off-chain)

A React + TypeScript SPA provides the user interface. It uses Web3.js to communicate with `window.ethereum` (MetaMask or any EIP-1193-compliant provider). The app reads on-chain state (balance, ERC-20 allowance, current chain ID) and constructs the calldata for each contract method before asking the user to sign the transaction.

2.3 Data Ingestion Layer

Recipients can be entered by hand or imported from CSV, plain-text, or Excel files. A shared `parseFile` utility normalises all three formats into a uniform in-memory list before the transaction is built. No file data is ever uploaded to a server; parsing happens entirely in the browser.

Without BatchIt	With BatchIt
N transactions for N recipients	1 transaction for N recipients
N gas payments (base fee × N)	1 gas payment (base fee × 1)
N wallet confirmations	1 wallet confirmation
N transaction hashes to audit	1 transaction hash to audit
Partial failure is invisible	Atomic: all succeed or all revert

3. Smart Contract Architecture

The BatchIt smart contract is minimalist by design. It acts solely as a dispatch mechanism — it does not custody funds, does not charge protocol fees, and does not maintain any on-chain storage (no mappings, no owner variable, no paused flag).

3.1 Contract Addresses

Celo Mainnet (Chain ID 42220)	0x634B0B2353D05e644d81C1Ab395f4b266E575830
Base Mainnet (Chain ID 8453)	0x0584aa5138E12275212C390E7B398fDb4B1c94B9

3.2 ABI & Function Signatures

batchTransfer(address token, address[] recipients, uint256[] amounts) payable

Transfers a different ERC-20 amount to each recipient. The caller must have pre-approved the contract for at least the sum of all amounts. The lengths of recipients and amounts must match; the contract will revert if they differ. Each amount is expressed in the token's native unit (i.e., wei-equivalent, accounting for the token's decimal places).

batchTransferEqual(address token, address[] recipients, uint256 amount) payable

Transfers an identical ERC-20 amount to every address in the recipients array. Conceptually equivalent to batchTransfer with a uniform amounts array, but uses a single uint256 parameter instead, which slightly reduces calldata size when the distribution is uniform.

batchTransferETH(address[] recipients, uint256[] amounts) payable

Distributes native coins (CELO on Celo, ETH on Base) to each recipient. The total msg.value sent with the transaction must equal the sum of all amounts; any excess is refunded or causes a revert depending on the contract implementation. The function does not handle ERC-20 tokens — the token parameter is omitted; for native ETH the token address is the zero address (0x000...000) as a UI convention.

3.3 ERC-20 Allowance Flow

Before any ERC-20 batch, the frontend calls the ERC-20 allowance(owner, spender) view function to check whether the user has already granted sufficient approval to the BatchIt contract. If the current allowance is less than the required total, a separate approve(spender, amount) transaction is submitted first. Only after that confirmation is the batch transaction sent.

This two-step flow (approve then batch) is standard ERC-20 practice and is transparent to the user: the UI shows a toast notification "Approving token..." followed by "Token approved!" before the "Processing transaction..." notification appears.

```
allowance = tokenContract.allowance(userAddress, batchContractAddress)
if (allowance < totalRequired) → approve(batchContract, totalRequired)
then → batchTransfer / batchTransferEqual (depending on mode)
```

4. Supported Networks & Tokens

4.1 Celo Mainnet

Celo (Chain ID 42220) is an EVM-compatible Layer 1 blockchain focused on mobile-first financial access and environmental sustainability. Block time is approximately 5 seconds, and gas prices are denominated in CELO (the native coin). Three tokens are supported out of the box:

Token	Contract Address
CELO (native)	0x471ece3750da237f93b8e339c536989b8978a438
cUSD (Celo Dollar)	0x765de816845861e75a25fca122bb6898b8b1282a
cEUR (Celo Euro)	0xD8763CBa276a3738E6DE85b4b3bF5FDed6D6cA73

4.2 Base Mainnet

Base (Chain ID 8453) is an Ethereum Layer 2 network incubated by Coinbase and built on the OP Stack. It inherits Ethereum's security while offering significantly lower transaction costs. Three tokens are supported:

Token	Contract Address
ETH (native)	(native coin — no contract address)
USDC	0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913
DAI	0x50c5725949A6F0c72E6C4a641F24049A917DB0Cb

4.3 Chain Switching

The ChainSelector component renders two clickable cards (one per network). On selection, the app calls `wallet_switchEthereumChain` via the EIP-1193 provider. If the user's wallet does not have the selected network configured, error code 4902 is caught and a human-readable message is shown prompting the user to add the network manually. After a successful switch, the window is reloaded to reset all React state to reflect the new chain context.

5. Transfer Modes

BatchIt offers three independent transfer modes, each exposed as a tab in the UI. The active tab determines which contract method is invoked and what form inputs are rendered.

5.1 Equal Amounts (`batchTransferEqual`)

The most common use case: send the same quantity of a token to every recipient. The user selects a token, enters a single amount (in human-readable units, e.g. "100" for 100 USDC), and provides a list of recipient addresses — one per line, either typed or imported from a file.

Internally, the frontend:

- Splits the textarea on newlines and trims whitespace from each line
- Validates each address with `web3.utils.isAddress()`
- Computes `totalAmountWei = amountWei × recipientCount`
- Calls `checkAndApproveToken` if the token is an ERC-20
- Calls `batchTransferEqual(token, recipients[], amountWei)`

5.2 Different Amounts (`batchTransfer`)

For payroll or distributions where each recipient receives a different quantity. The user specifies a token and then fills in address/amount pairs — either via the dynamic form (add row buttons) or by importing a two-column CSV/Excel file (address in column A, amount in column B).

Internally:

- The frontend collects all valid (address, amount) pairs
- Converts each amount to wei
- Sums all weis for the approval step
- Calls `batchTransfer(token, recipients[], amounts[])`

5.3 Native Transfer (`batchTransferETH`)

For distributing native coins (CELO on Celo, ETH on Base) without going through an ERC-20 approve step. The user provides address/amount pairs identical to the "Different Amounts" form.

Because native coins do not require ERC-20 approval, there is no approve transaction. The total value is computed and passed as `msg.value` in a single call to `batchTransferETH(recipients[], amounts[])`.

6. File Import System

A distinguishing feature of BatchIt is its ability to ingest recipient lists from files, dramatically accelerating the workflow for operators managing large distributions.

6.1 Supported File Formats

- CSV (.csv) — comma-separated values; address in column 1, optional amount in column 2
- Plain text (.txt) — same structure as CSV; one entry per line
- Excel (.xlsx / .xls) — parsed with the SheetJS (xlsx) library; first worksheet, row 1 onwards
- PDF files are explicitly rejected with a user-facing info toast ("PDF files are not supported yet.")

6.2 Parse Logic

All three parseable formats are handled by a single shared parseFile function. The function:

- Detects whether the file is Excel by extension; if so, reads it as an ArrayBuffer and uses XLSX.read()
- Otherwise reads it as a UTF-8 text string and splits on newlines, then on commas
- Maps each row to either a bare address string (Equal Amounts tab) or an { address, amount } object (other tabs)
- Filters out empty or null rows
- Merges the parsed list into existing state without resetting already-entered manual entries
- Shows a success toast reporting how many recipients were imported, or a warning if the file was empty or unparseable

6.3 Privacy Model

All file parsing is performed entirely in the browser using the FileReader API. No file contents are transmitted to any server. BatchIt has no backend; there is nothing to receive or store user data.

7. Frontend Architecture

The BatchIt frontend is a React 18 application written in TypeScript, bundled with Vite 5, and styled with Tailwind CSS 3. It is structured around two React Contexts and three primary UI components.

7.1 Technology Stack

Layer	Technology
UI Framework	React 18 + TypeScript
Build Tool	Vite 5
Styling	Tailwind CSS 3
Icons	lucide-react
Blockchain	Web3.js 4
Spreadsheet Parsing	SheetJS (xlsx 0.18)
Linting	ESLint 9 + typescript-eslint

7.2 Web3Context

The Web3Context is the application's single source of truth for all blockchain state. It exposes:

- `web3` — the Web3 instance wrapping `window.ethereum`
- `account` — the currently connected address (or null)
- `balance` — the native coin balance formatted to 4 decimal places
- `chainId` — the current chain ID as a decimal integer
- `selectedChain` — the user's active network choice ("celo" | "base")
- `connectWallet()` — requests `eth_requestAccounts`, sets `account`, `balance`, `chainId`
- `disconnectWallet()` — clears local state (note: MetaMask does not expose a programmatic disconnect)
- `switchChain(chain)` — calls `wallet_switchEthereumChain`, catches error 4902
- `isConnecting` — boolean loading flag during the connect flow

The context also exports the static CONFIG constants (`CONTRACT_ADDRESSES`, `CHAIN_IDS`, `TOKENS`) so they are available anywhere in the component tree without re-importing the config file.

7.3 ToastContext

A lightweight notification context that renders transient feedback messages at the top-right of the viewport. The `showToast(message, type, durationMs)` function accepts four types: "success", "error", "info", and "warning". Default duration is 4 seconds; transaction hash confirmations use 8 seconds to give users time to copy the hash.

7.4 Component Breakdown

WalletConnect

Renders a top-of-page bar. When no wallet is connected it shows a single "Connect Wallet" button. When connected it displays a truncated account address (first 6 + last 4 characters), the native balance, and a "Disconnect" button.

ChainSelector

Two card buttons (Celo and Base) with visual state: the active network card has a coloured border and a pulsing green dot. Beneath the cards, the currently deployed contract address is displayed in monospace so advanced users can verify it independently.

TransferForms

The most complex component. Hosts the three-tab system and all form state. Each tab renders its own form with the appropriate fields. The FileUpload sub-component is a reusable drag-and-drop / click-to-upload area rendered in all three tabs with per-tab configuration.

8. User Journey

The following describes the complete end-to-end journey of a user performing a batch ERC-20 distribution.

Step 1: Open the Application

The user navigates to the BatchIt URL. The page loads with a dark slate/blue gradient background, the BatchIt logo (a paper-plane icon), and the tagline "Send tokens to multiple recipients across Celo and Base networks in a single transaction." The page is fully responsive and functional on mobile browsers with MetaMask Mobile installed.

Step 2: Connect Wallet

The user clicks the "Connect Wallet" button in the WalletConnect bar. The MetaMask extension (or mobile app) prompts for account access. On approval, the bar updates to show the connected address and native balance. A green success toast appears: "Wallet connected successfully."

Step 3: Select Network

The user clicks either the Celo or Base card in the ChainSelector section. If the wallet is already on the target network, the card highlights immediately. If not, MetaMask prompts to switch networks. The contract address for the selected network is displayed below the cards.

Step 4: Choose Transfer Mode

The user selects one of three tabs in the TransferForms panel: Equal Amounts, Different Amounts, or Native Transfer. The form fields update accordingly.

Step 5: Configure the Transfer

For Equal Amounts: the user selects a token from the dropdown, enters the per-recipient amount, and either types recipient addresses (one per line) or clicks the file upload area to import a CSV/Excel.

For Different Amounts: the user selects a token, then either fills in address/amount rows manually (clicking "+ Add Recipient" for each) or imports a two-column file.

For Native Transfer: same as Different Amounts but no token selection is needed.

Step 6: Submit

The user clicks the "Send" button. If the token requires approval, an "Approving token..." toast appears and MetaMask prompts for the approve transaction. After

confirmation, a "Token approved!" toast appears, followed by "Processing transaction..." and a second MetaMask prompt for the batch transaction itself.

Step 7: Confirmation

On success, a toast displays: "Transaction successful! 0x<txHash>" and remains visible for 8 seconds so the user can copy the hash. The form fields reset to their empty initial state, ready for another batch.

On failure, the error message from the contract revert or from web3 is displayed in a red error toast.

9. Security Model

BatchIt is built with a minimal-trust, minimal-footprint philosophy. There are several explicit security properties to understand.

9.1 Contract Is Non-Custodial

The smart contract never holds funds on behalf of users. ERC-20 approvals grant the contract permission to pull tokens during a specific call; no tokens remain in the contract after the call completes. Native coin sent as `msg.value` is fully distributed to recipients within the same transaction; any excess beyond the declared sum is handled per the contract implementation.

9.2 Address Validation

Every recipient address entered manually is validated with `web3.utils.isAddress()` before the transaction is built. Invalid addresses cause an error toast and abort the submission without spending gas. Addresses imported from files go through the same validation pass.

9.3 Client-Side Only

BatchIt has no backend server, no database, and no analytics. All computation happens locally in the browser. Files uploaded for parsing never leave the user's machine. Private keys never leave MetaMask. The application is a static bundle served over HTTPS.

9.4 Open Verification

The contract address for each network is displayed prominently in the ChainSelector component. Users can independently verify the contract bytecode and ABI on Celoscan (for Celo) and Basescan (for Base) before approving any token.

9.5 Wallet Event Handling

The Web3Context registers listeners for `accountsChanged` and `chainChanged` events from the MetaMask provider. Account changes update the displayed address and balance. Chain changes trigger a `window.location.reload()` to ensure all chain-dependent state is refreshed cleanly rather than becoming stale or mismatched.

9.6 Considerations & Limitations

- BatchIt relies on the MetaMask EIP-1193 provider; it does not support WalletConnect or hardware wallets directly in the current version.

- Large batches (hundreds of recipients) may approach the block gas limit on some networks. The contract does not currently split large batches automatically.
- The "disconnectWallet" button clears application state but does not revoke MetaMask's connection to the site; the user must do that in MetaMask's connected sites panel.
- ERC-20 infinite approvals are not used; the approval amount is set to exactly the total required for the current batch.

10. Gas Economics

One of BatchIt's core value propositions is gas efficiency. The following analysis compares batch vs. sequential approaches.

10.1 Why Batching Saves Gas

An Ethereum/EVM transaction has a base cost of 21,000 gas regardless of what it does. Sending N sequential transfers incurs this overhead N times. A batch transaction incurs it once, then pays per-recipient overhead that is lower than a full transaction cost because the contract's iteration logic reuses already-loaded storage and code.

For ERC-20 transfers, each individual `token.transfer()` costs roughly 30,000–65,000 gas. In a batch, the contract loops over the recipient array and calls the token's `transferFrom()` for each entry. The per-recipient cost inside the loop is lower than a standalone transaction because the ERC-20 contract's code and storage slots (total supply, allowances) are already warm from the first access.

10.2 Approximate Savings (Illustrative)

Recipients	Sequential (approx. gas)		
10	650,000	120,000	~82%
50	3,250,000	440,000	~86%
100	6,500,000	850,000	~87%
200	13,000,000	1,650,000	~87%

Note: figures are illustrative and depend on token implementation, current block base fee, and network conditions. Celo's native fee currency flexibility means gas can be paid in cUSD or cEUR rather than CELO, which can be advantageous for treasuries holding stablecoins.

10.3 Equal vs. Variable Amounts

`batchTransferEqual` is slightly cheaper in calldata than `batchTransfer` for large lists because it encodes only one `uint256` amount instead of an array of N amounts. The calldata savings are proportional to the recipient count: for 100 recipients, this can save several thousand gas units.

11. Use Cases

11.1 DAO / Protocol Treasury Payroll

Decentralized autonomous organizations routinely pay contributors in governance tokens or stablecoins at the end of a grant period. BatchIt reduces what was a multi-hour, multi-confirmation process to a single transaction that a multisig can sign in one shot.

11.2 Token Airdrops

Projects launching new tokens or rewarding early users can upload a CSV of eligible addresses and amounts generated off-chain (e.g., from a Merkle snapshot) and execute the entire airdrop in one or a few batches. The Equal Amounts mode is ideal for flat-rate distributions; Different Amounts accommodates merit-based or tiered allocations.

11.3 Campaign Reward Distribution

Blockchain incentive campaigns (trading competitions, liquidity mining, referral programs) generate daily or weekly reward lists. BatchIt fits naturally into the settlement step of such pipelines: generate the CSV from the campaign database, upload, review, send.

11.4 Payroll for Remote Teams

Teams paid in stablecoins (cUSD, USDC, DAI) can use BatchIt to settle monthly payroll. The finance operator exports a payroll spreadsheet with wallet addresses and amounts, imports it into BatchIt, and executes. One transaction, one hash, one line in the treasury ledger.

11.5 NFT Royalty Distribution

Projects that aggregate royalties in a treasury wallet and distribute periodically to a fixed set of creators or revenue-share holders can automate the distribution step with BatchIt — either equal splits or proportional shares.

11.6 Grant Programs

Ecosystem grant programs on Celo or Base can disburse tranches to multiple grantees simultaneously, reducing administrative overhead and on-chain transaction count.

12. Roadmap & Future Work

12.1 Short-Term (v1.1)

- WalletConnect v2 integration — support hardware wallets, mobile wallets beyond MetaMask
- Gas estimation preview — show estimated gas cost and USD equivalent before submission
- Auto-batch splitting — detect when a list exceeds the safe gas limit and automatically split into multiple transactions
- Transaction history — local storage log of past batches with status and hash

12.2 Medium-Term (v1.2 - v2.0)

- Additional network support — Polygon, Arbitrum, Optimism, Avalanche C-Chain
- CSV export of transaction results — post-send export of address, amount, and tx hash for accounting
- Address book — save frequently used recipient lists with names
- Merkle airdrop mode — for very large distributions (10,000+ recipients) using a Merkle tree contract pattern where recipients claim rather than receiving a push
- Multi-token batch — send different tokens to different recipients in a single compound transaction

12.3 Long-Term

- Gasless mode — relay transactions through a meta-transaction service so recipients with no native coin can still receive tokens
- Scheduled batches — queue a batch for execution at a future block or time via a keeper network
- On-chain analytics — a read-only dashboard showing historical batches by contract address

13. Conclusion

BatchIt addresses a real and recurring pain point in on-chain operations: the prohibitive cost and complexity of distributing tokens to many recipients one at a time. By routing multi-recipient distributions through a purpose-built smart contract, BatchIt delivers dramatic reductions in gas cost, operational time, and audit complexity.

The platform is accessible to non-technical operators through its file-import system and guided wallet flow, while remaining transparent and auditable for developers through its open ABI, verifiable contract addresses, and client-side-only architecture. Support for both Celo and Base Mainnet from day one ensures the tool is useful across two of the most active EVM ecosystems for payments and DeFi.

BatchIt is ready to serve DAOs, project teams, grant programs, marketing campaigns, and any operator who regularly needs to move tokens to many addresses at once. It is designed to be fast to use, easy to audit, and safe by construction.